e6cc8b77-da2f-48c0-a212-ce50a0094345

<current>

<request>

You are working on an issue in the 'animeshkundu/youtube-audio' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/youtube-audio/youtube-audio (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-26T01:50:50.290Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Role: Principal AI Architect & Repository Scaffolder

Objective
Your goal is to fundamentally restructure and scaffold this repository (whether currently empty or existing) to be fully **"AI-Enabled."** "AI-Enabled" is defined as a repository optimized for **Agentic Coding** within the GitHub portal, where multiple AI agents work in parallel. The structure must enforce a strict workflow where documentation drives code, testing is mandatory, and agents act autonomously using internet research and self-validation.

Phase 1: Context & Analysis
1. **Analyze the Current State:**
    * If this is an **existing repo**: Read the current codebase to understand the language, framework, and business logic. Do not delete code, but move documentation into the new structure if necessary.
    * If this is a **new/empty repo**: Assume a "Greenfield" setup but prepare the structure to be agnostic to the specific language until specified (or default to a modern stack like TypeScript/Rust/Python if you need to create examples).

```
## Phase 2: Structural Scaffolding (Execute Immediately)
You must create the following directory structure and files. If
they exist, update them to match these strict requirements.

### 1. The `docs/` Hierarchy (The Brain)
Create a `docs/` folder with the following sub-directories.
Inside each, create a `README.md` explaining its purpose:
* `docs/adrs/`: For Architecture Decision Records. Create a
template `0000-template.md`.
* `docs/specs/`: For Technical Specifications. Agents MUST write
a spec here *before* writing code.
* `docs/architecture/`: For system diagrams (Mermaid.js) and
high-level design.
* `docs/history/`: For recording "Handoffs" and deprecated
logic.
* `docs/agent-instructions/`: (See Section 2 below).

### 2. The Agent Instruction Layer (The Protocols)
Create `docs/agent-instructions/` and populate it with the
following critical files:

**`docs/agent-instructions/00-core-philosophy.md`**
* **Docs = Code:** Explicitly state that no code is written
without updating `docs/specs/` first.
* **Sync:** Instructions on how to update `docs/history/` after
work is complete.
* **The CEO Model:** Explain that the initiating agent is the
"CEO" and sub-agents are workers.
* **First Principles:** Reasoning, deep thinking, and step-by-
step planning are required.

**`docs/agent-instructions/01-research-and-web.md`**
* **Internet is First-Class:** Agents MUST use web search to
find current best practices before coding.
* **Validation:** Use search to verify library versions and
patterns.
* **Saturation:** Research until information saturation is
reached before implementation.

**`docs/agent-instructions/02-testing-and-validation.md`**
* **90% Rule:** Mandate a strict minimum of 90% code coverage
for Unit and Integration tests.
* **Test-Driven:** Agents must write the test *before* or *with*
the implementation.
* **Self-Correction:** Instructions on how an agent must run
tests locally (or create a script to do so) to verify their own
```

```
work before committing.

**`docs/agent-instructions/03-tooling-and-pipelines.md`**
* **Tool Creation:** Instruct agents that if they perform a
verification task twice, they *must* write a script (shell/
python) for it.
* **CI/CD:** Instructions on prioritizing GitHub Actions for
Lint, Build, Test, and Deploy.

### 3. Root Context Files (The System Prompt)
Create or overwrite the following files in the root:

**`.github/copilot-instructions.md`** &amp;amp; **`claude.md`**
(Content should be identical/synced):
* Write a comprehensive system prompt that references the `docs/
` folder.
* **Instruction:** "Before answering any request, you MUST read
`docs/agent-instructions/`."
* **Instruction:** "Check `docs/adrs/` for past decisions to
avoid regression."
* **Instruction:** "If you modify code, you MUST update the
corresponding Spec and Architecture doc."
* **Instruction:** "If you are unsure, search the internet. Do
not hallucinate APIs."

### 4. Agent Configuration Folders
* Create `.github/agents/` (Placeholder for future GitHub-native
agent configs).
* Create `.claude/agents/` (Placeholder for Claude-specific
configurations).

## Phase 3: Automation &amp;amp; Harnessing
1.  **Scaffold CI/CD:**
    * Create a `.github/workflows/ci.yml` file.
    * **Prompt the agent to research:** "Search for the best
GitHub Actions workflow for [Language of Repo] that includes
linting, testing with coverage reporting, and security
scanning."
    * Implement that workflow.
2.  **Scaffold Validation Scripts:**
    * Create a `scripts/` or `tools/` folder.
    * Create a `validate.sh` (or equivalent) that runs the
linter and test suite.
    * **Reasoning:** This allows any agent to simply run `./
scripts/validate.sh` to check their work.

## Execution Plan for You (The LLM)
```

```
1.  **Think:** Review the requirements above.
2.  **Research:** Search the internet for "best
practices .github/copilot-instructions.md structure" and
"Mermaid.js architecture diagrams for [Language]" to ensure the
templates you generate are top-tier.
3.  **Act:** Generate the file structure and content described
above.
4.  **Verify:** Ensure that the `copilot-instructions.md`
explicitly links the behavior of the AI to the files in `docs/`.

**GO.** Start by listing the file structure you are about to
create, then generate the content for the critical documentation
and instruction files.
</problem_statement>
```

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

<prompt>

```
# Role: Principal AI Architect & Repository Scaffolder

## Objective
Your goal is to fundamentally restructure and scaffold this
repository (whether currently empty or existing) to be fully **"AI-
Enabled."** "AI-Enabled" is defined as a repository optimized for
**Agentic Coding** within the GitHub portal, where multiple AI
agents work in parallel. The structure must enforce a strict
workflow where documentation drives code, testing is mandatory, and
agents act autonomously using internet research and self-validation.

## Phase 1: Context & Analysis
1.  **Analyze the Current State:**
    * If this is an **existing repo**: Read the current codebase to
understand the language, framework, and business logic. Do not
delete code, but move documentation into the new structure if
necessary.
    * If this is a **new/empty repo**: Assume a "Greenfield" setup
but prepare the structure to be agnostic to the specific language
until specified (or default to a modern stack like TypeScript/Rust/
Python if you need to create examples).

## Phase 2: Structural Scaffolding (Execute Immediately)
You must create the following directory structure and files. If they
```

```
exist, update them to match these strict requirements.

### 1. The `docs/` Hierarchy (The Brain)
Create a `docs/` folder with the following sub-directories. Inside
each, create a `README.md` explaining its purpose:
* `docs/adrs/`: For Architecture Decision Records. Create a template
`0000-template.md`.
* `docs/specs/`: For Technical Specifications. Agents MUST write a
spec here *before* writing code.
* `docs/architecture/`: For system diagrams (Mermaid.js) and high-
level design.
* `docs/history/`: For recording "Handoffs" and deprecated logic.
* `docs/agent-instructions/`: (See Section 2 below).

### 2. The Agent Instruction Layer (The Protocols)
Create `docs/agent-instructions/` and populate it with the following
critical files:

**`docs/agent-instructions/00-core-philosophy.md`**
* **Docs = Code:** Explicitly state that no code is written without
updating `docs/specs/` first.
* **Sync:** Instructions on how to update `docs/history/` after work
is complete.
* **The CEO Model:** Explain that the initiating agent is the "CEO"
and sub-agents are workers.
* **First Principles:** Reasoning, deep thinking, and step-by-step
planning are required.

**`docs/agent-instructions/01-research-and-web.md`**
* **Internet is First-Class:** Agents MUST use web search to find
current best practices before coding.
* **Validation:** Use search to verify library versions and
patterns.
* **Saturation:** Research until information saturation is reached
before implementation.

**`docs/agent-instructions/02-testing-and-validation.md`**
* **90% Rule:** Mandate a strict minimum of 90% code coverage for
Unit and Integration tests.
* **Test-Driven:** Agents must write the test *before* or *with* the
implementation.
* **Self-Correction:** Instructions on how an agent must run tests
locally (or create a script to do so) to verify their own work
before committing.

**`docs/agent-instructions/03-tooling-and-pipelines.md`**
* **Tool Creation:** Instruct agents that if they perform a
```

```
verification task twice, they *must* write a script (shell/python)
for it.
* **CI/CD:** Instructions on prioritizing GitHub Actions for Lint,
Build, Test, and Deploy.

### 3. Root Context Files (The System Prompt)
Create or overwrite the following files in the root:

**`.github/copilot-instructions.md`** & **`claude.md`** (Content
should be identical/synced):
* Write a comprehensive system prompt that references the `docs/`
folder.
* **Instruction:** "Before answering any request, you MUST read
`docs/agent-instructions/`."
* **Instruction:** "Check `docs/adrs/` for past decisions to avoid
regression."
* **Instruction:** "If you modify code, you MUST update the
corresponding Spec and Architecture doc."
* **Instruction:** "If you are unsure, search the internet. Do not
hallucinate APIs."

### 4. Agent Configuration Folders
* Create `.github/agents/` (Placeholder for future GitHub-native
agent configs).
* Create `.claude/agents/` (Placeholder for Claude-specific
configurations).

## Phase 3: Automation & Harnessing
1.  **Scaffold CI/CD:**
    * Create a `.github/workflows/ci.yml` file.
    * **Prompt the agent to research:** "Search for the best GitHub
Actions workflow for [Language of Repo] that includes linting,
testing with coverage reporting, and security scanning."
    * Implement that workflow.
2.  **Scaffold Validation Scripts:**
    * Create a `scripts/` or `tools/` folder.
    * Create a `validate.sh` (or equivalent) that runs the linter
and test suite.
    * **Reasoning:** This allows any agent to simply run `./scripts/
validate.sh` to check their work.

## Execution Plan for You (The LLM)
1.  **Think:** Review the requirements above.
2.  **Research:** Search the internet for "best practices .github/
copilot-instructions.md structure" and "Mermaid.js architecture
diagrams for [Language]" to ensure the templates you generate are
top-tier.
```

```
3.  **Act:** Generate the file structure and content described
above.
4.  **Verify:** Ensure that the `copilot-instructions.md` explicitly
links the behavior of the AI to the files in `docs/`.

**GO.** Start by listing the file structure you are about to create,
then generate the content for the critical documentation and
instruction files.
```